

1. Fundamentals of errors and debugging:

- Error

An error is a mistake made by the programmer while writing the code. It can happen because of typing the wrong symbol, using the wrong logic, or forgetting a part of the code. If an error is not corrected, the program may not work as expected.

Example: A programmer writes $a + b$ instead of $a * b$ when the task is to multiply two numbers.

- Bug

A bug is a problem in a program that appears because of one or more errors in the code. It causes the program to give wrong results, crash, or behave differently from what the programmer intended.

Example: A calculator program displays the sum of two numbers instead of their product because the wrong operator was used.

- Debugging

Debugging is the process of finding the cause of bugs and fixing them. During debugging, the programmer carefully checks the code, identifies the mistake, corrects it, and tests the program again to make sure it works properly.

Example: If a multiplication program is giving addition as the output, the programmer checks the code, changes $+$ to $*$, and tests the program to verify the correct result.

2. Classifications of Errors in C

- **Syntax error**

A syntax error happens when the program does not follow the grammar or rules of the C programming language. The compiler detects these errors before the program runs, so the code must be corrected before compilation can succeed.

Primary Cause: Missing symbols, spelling mistakes in keywords, or unmatched brackets.

Example: `int a = 10` (missing semicolon ;)

- **Compilation/Type errors**

A compilation or type error occurs when incompatible data types are used or when variables or functions are not declared properly. The compiler identifies these errors during compilation.

Primary Cause: Using the wrong data type, assigning incompatible values, or using undeclared variables.

Example: `int *p = "hello";` (string assigned to an integer pointer)

- **Linker errors**

A linker error occurs after the program is compiled, when the linker tries to combine different object files into one executable. If a required function or library is missing, the program cannot be created.

Primary Cause: Declaring a function without defining it or forgetting to include required libraries.

Example: undefined reference to 'main' (linker cannot find the main() function)

- Runtime Errors

A runtime error occurs while the program is running. The program compiles successfully, but it crashes or behaves abnormally because of an invalid operation.

Primary Cause: Dividing by zero, accessing invalid memory, or other illegal operations during execution.

Example: `int x = 10 / 0;` (division by zero causes the program to fail)

- Logical Errors

A logical error is the most difficult type of error because the program compiles and runs without crashing, but it produces incorrect results. This happens when the programmer uses the wrong logic or algorithm.

Primary Cause: Incorrect formulas, wrong conditions, or mistakes in the program's logic.

Example: `for(i = 0; i < 5; i++)` (loop runs from 0 to 4, correct condition is `0 <= 5`)

Why Does a Program Compile Successfully but Produce Incorrect Output?

A program can compile successfully but still produce incorrect output because the compiler only checks whether the code follows the rules of the C language (syntax and type rules). It cannot determine whether the program's logic is correct or whether it solves the problem as intended.

ERROR 1: Syntax error

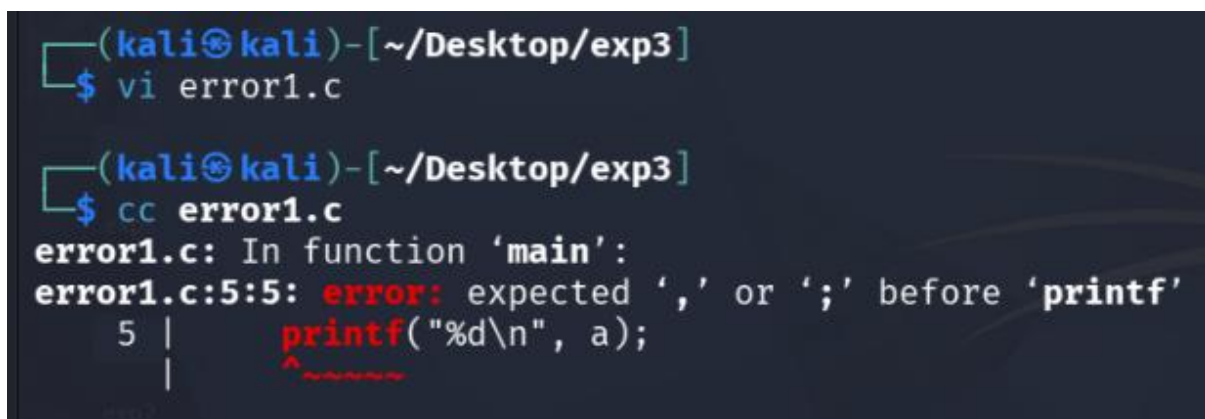
//error1.c (code with bug)

```
#include <stdio.h>

int main()
{
    int a = 10                //line with error

    printf("%d\n", a);

    return 0;
}
```



```
(kali㉿kali)-[~/Desktop/exp3]
$ vi error1.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc error1.c
error1.c: In function 'main':
error1.c:5:5: error: expected ',' or ';' before 'printf'
   5 |     printf("%d\n", a);
     |     ^~~~~~
```

Breakdown of Error Message

- Filename: sample1.c
- Line & Column: Line 5, Column 5
- Type of Message: error
- Description: Expected , or ; before printf statement.

Error occurred due to a missing “ , ” or “ ; ” in the code.

CORRECT CODE:

```
#include <stdio.h>
```

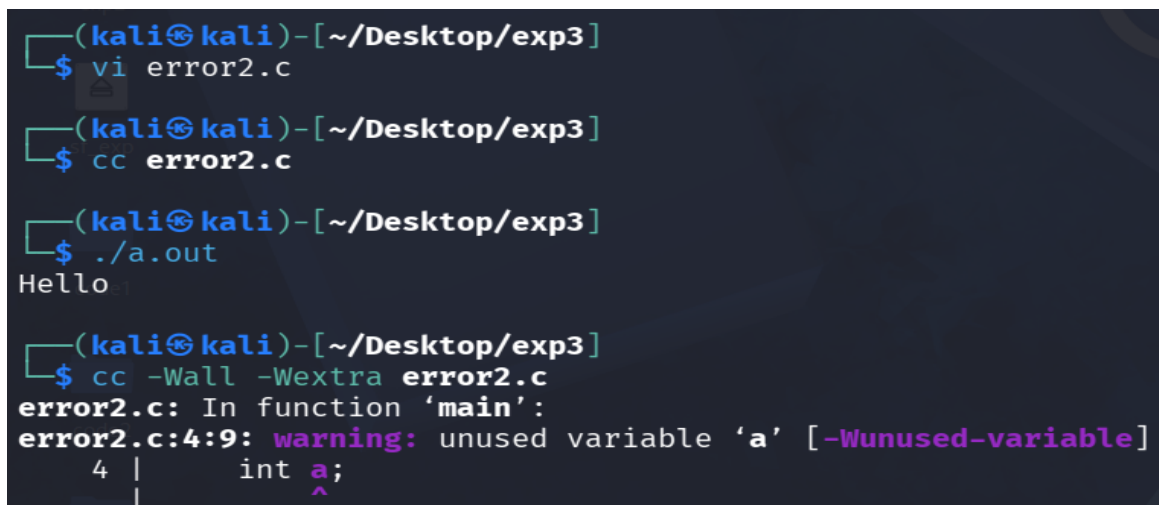
```
int main()
{
    int a = 10;                //error resolved
    printf("%d\n", a);
    return 0;
}
```

ERROR 2: Compiler warnings

```
//error2.c (code with a warning)

#include <stdio.h>

int main()
{
    int a;                    //warning line
    printf("Hello\n");
    return 0;
}
```



```
(kali㉿kali)-[~/Desktop/exp3]
$ vi error2.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc error2.c

(kali㉿kali)-[~/Desktop/exp3]
$ ./a.out
Hello

(kali㉿kali)-[~/Desktop/exp3]
$ cc -Wall -Wextra error2.c
error2.c: In function 'main':
error2.c:4:9: warning: unused variable 'a' [-Wunused-variable]
    4 |     int a;
      |         ^
```

An unused variable is found that was extra in the program.

(int a) was not assigned any value and was not used in the execution of the program therefore producing a compiler warning on using Wall and Wextra.

CORRECT CODE:

```
#include <stdio.h>

int main()
{
    printf("Hello\n");
    return 0;
}
```

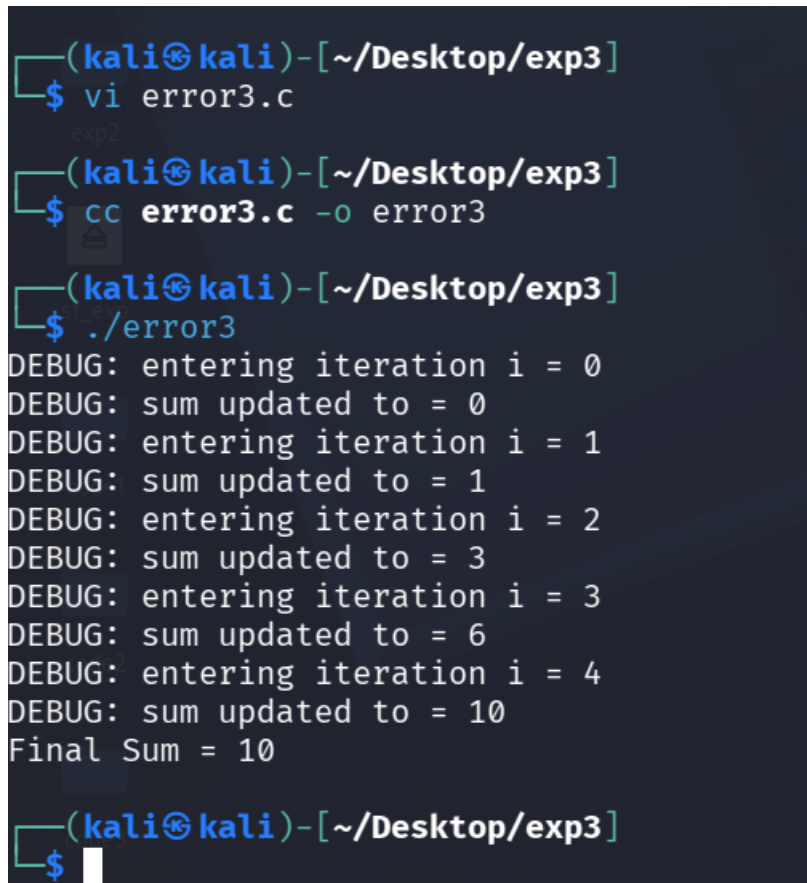
ERROR 3: printf-Based Debugging (Logical bug)

//error3.c (code with a logical bug)

```
#include <stdio.h>

int main(void)
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
    }
}
```

```
    printf("DEBUG: sum updated to = %d\n", sum);  
}  
printf("Final Sum = %d\n", sum);  
return 0;  
}
```



```
(kali㉿kali)-[~/Desktop/exp3]  
$ vi error3.c  
exp2  
(kali㉿kali)-[~/Desktop/exp3]  
$ cc error3.c -o error3  
(kali㉿kali)-[~/Desktop/exp3]  
$ ./error3  
DEBUG: entering iteration i = 0  
DEBUG: sum updated to = 0  
DEBUG: entering iteration i = 1  
DEBUG: sum updated to = 1  
DEBUG: entering iteration i = 2  
DEBUG: sum updated to = 3  
DEBUG: entering iteration i = 3  
DEBUG: sum updated to = 6  
DEBUG: entering iteration i = 4  
DEBUG: sum updated to = 10  
Final Sum = 10  
(kali㉿kali)-[~/Desktop/exp3]  
$
```

Expected output: 15

Actual output: 10

The trace shows $i = 0, 1, 2, 3$, and 4. If the intended calculation is the sum from 1 to 5, the loop range is wrong. Start at 1 and include 5.

CORRECT CODE:

```
#include <stdio.h>

int main(void)
{
    int i, sum = 0;
    for(i = 1; i <= 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("Final Sum = %d\n", sum);
    return 0;
}
```

This correct code returns the output value as 15.

3. Introduction to GDB (GNU Debugger)

- GNU

GNU is a free and open-source software project started by **Richard Stallman in 1983**. Its goal was to create a complete operating system that works like Unix but is freely available for everyone to use, modify, and share. Today, many Linux operating systems use GNU tools and utilities.

Example: The GCC compiler and GDB debugger are popular software developed as part of the GNU project.

- GDB (GNU Debugger)

GDB (GNU Debugger) is a tool used by programmers to find and fix errors in their programs. It helps developers run a program step by step, check the values of variables, set breakpoints to pause execution, and identify the exact location where a program crashes or behaves incorrectly.

Example: If a C program crashes because of a segmentation fault, a programmer can use GDB to locate the faulty line of code and determine the cause of the error.

ERROR 4: Debugging a Logical Error using GDB

//error4.c (code with a logic bug using GDB)

```
#include <stdio.h> // Line 1
int main() // Line 2
{ // Line 3
    int i, sum = 0; // Line 4
    for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
    { // Line 6
        sum = sum + i; // Line 7
    } // Line 8
    printf("Sum = %d\n", sum); // Line 9
    return 0; // Line 10
}
```

```
(kali㉿kali)-[~/Desktop/exp3]
$ vi error4.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error4.c -o error4

(kali㉿kali)-[~/Desktop/exp3]
$ gdb ./error4
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./error4 ...
(gdb) list
1      #include <stdio.h>                                // Line 1
2      int main()                                         // Line 2
3      {                                                 // Line 3
4          int i, sum = 0;                                // Line 4
5          for(i = 1; i < 5; i++)                        // Line 5 (Logical Bug: should be i ≤ 5)
6          {                                             // Line 6
7              sum = sum + i;                            // Line 7
8          }                                             // Line 8
9          printf("Sum = %d\n", sum);                   // Line 9
10         return 0;                                     // Line 10
(gdb) break 7
Breakpoint 1 at 0x1151: file error4.c, line 7.
```

```

Breakpoint 1 at 0x1151: file error4.c, line 7.
(gdb) run
Starting program: /home/kali/Desktop/exp3/error4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error4.c:7
7          sum = sum + i;          // Line 7
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at error4.c:7
7          sum = sum + i;          // Line 7
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at error4.c:7
7          sum = sum + i;          // Line 7
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at error4.c:7
7          sum = sum + i;          // Line 7
1: i = 4
2: sum = 6
(gdb) continue
Continuing.

```

```

Continuing.
Sum = 10
[Inferior 1 (process 70840) exited normally]
(gdb) █

```

Expected outcome: 15 (1+2+3+4+5)

Actual outcome: 10 (1+2+3+4)

The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition should be changed from $i < 5$ to $i \leq 5$.

CORRECT CODE:

```
#include <stdio.h> // Line 1
int main() // Line 2
{ // Line 3
    int i, sum = 0; // Line 4
    for(i = 1; i <= 5; i++) // Line 5 (Logical Bug resolved)
    { // Line 6
        sum = sum + i; // Line 7
    } // Line 8
    printf("Sum = %d\n", sum); // Line 9
    return 0; // Line 10
}
```

This returns the output as 15.

4. Debugging Functions (next vs. step)

- next Command

The next command in GDB executes the current line of code and then moves to the next line. If the current line contains a function call, GDB runs the entire function automatically and does not enter the function's code. This command is useful when you trust that the function is working correctly and want to continue debugging the main program.

Example:

If the current line is:

```
Sum = add(a, b);
```

Using next will execute the add() function completely and stop at the line after it.

- step Command

The step command also executes the current line, but if the line contains a function call, GDB enters the function and allows you to execute its code one line at a time. This helps you examine how the function works and find errors inside it.

Example:

If the current line is:

```
sum = add(a, b);
```

Using step will take you inside the add() function so you can inspect each statement.

ERROR 5: Debugging Functions (next vs. step)

//error5.c (debugging code using next and step)

```
#include <stdio.h>
```

```
int square(int x)
```

```
{
```

```
    int result1;
```

```
    result1 = x * x;
```

```
    return result1;
```

```
}
```

```
int main()
```

```
{
```

```
    int result, n = 5;
```

```
    result = square(n);
```

```
    printf("Square = %d\n", result);
```

```
    return 0;
```

```
}
```

Using NEXT:

```
(kali㉿kali)-[~/Desktop/exp3]
$ vi error5.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error5.c -o error5

(kali㉿kali)-[~/Desktop/exp3]
$ gdb ./error5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./error5...
(gdb) break main
Breakpoint 1 at 0x1156: file error5.c, line 10.
(gdb) run
Starting program: /home/kali/Desktop/exp3/error5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error5.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) next
12      printf("Square = %d\n", result);
(gdb) print result
$1 = 25
```

If you stop at the line:

result = square(n);

and type:

(gdb) next

GDB executes the entire square() function without showing its internal statements. It then moves directly to:

printf("Square = %d\n", result);

If you print the value:

(gdb) print result

Output: \$1 = 25

Using STEP:

```
(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error5.c -o error5

(kali㉿kali)-[~/Desktop/exp3]
$ gdb ./error5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./error5 ...
(gdb) break main
Breakpoint 1 at 0x1156: file error5.c, line 10.
(gdb) run
Starting program: /home/kali/Desktop/exp3/error5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error5.c:10
10      int result, n = 5;
```

```
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at error5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result 1
✗ No symbol "result" in current context.
(gdb) print result1
$2 = 25
(gdb)
```

If you stop at the same line:

result = square(n);

and type:

(gdb) step

GDB enters the square() function and allows you to execute it line by line:

```
int result1;  
result1 = x * x;  
return result1;
```

You can inspect variables:

```
(gdb) print x
```

```
$1 = 5
```

```
(gdb) next
```

```
(gdb) print result1
```

```
$2 = 25
```

There is no error in the given program. The step and next commands in GDB are not used to automatically detect errors. Instead, they help the programmer trace program execution. The step command enters a function and executes it line by line, making it useful for finding logical or runtime errors inside the function. The next command executes the function completely and moves to the next line without entering the function body, making it useful when the function is already known to work correctly.

ERROR6: Debugging Runtime Errors using Backtrace (backtrace / bt)

//error6.c (debugging using bt)

```
#include <stdio.h>

int divide(int a, int b)
{
    return a / b;                // Division by zero when b = 0
}

int calculate(int x)
{
    return divide(x, 0);         // Passes 0 as divisor
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

```

(kali㉿kali)-[~/Desktop/exp3]
$ vi error6.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error6.6 -o error6
/usr/bin/x86_64-linux-gnu-ld.bfd: cannot find error6.6: No such file or directory
collect2: error: ld returned 1 exit status

(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error6.c -o error6

(kali㉿kali)-[~/Desktop/exp3]
$ gdb ./error6
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./error6 ...
(gdb) run
Starting program: /home/kali/Desktop/exp3/error6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000555555555147 in divide (a=10, b=0) at error6.c:4
4      return a / b;           // Division by zero when b = 0

```

```

(gdb) backtrace
#0  0x0000555555555147 in divide (a=10, b=0) at error6.c:4
#1  0x0000555555555166 in calculate (x=10) at error6.c:8
#2  0x000055555555517a in main () at error6.c:12
(gdb) frame 0
#0  0x0000555555555147 in divide (a=10, b=0) at error6.c:4
4      return a / b;           // Division by zero when b = 0
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x0000555555555166 in calculate (x=10) at error6.c:8
8      return divide(x, 0);    // Passes 0 as divisor
(gdb) print x
$2 = 10
(gdb) █

```

The program crashes because of division by zero.

Explanation of the Backtrace:

- #0 divide() The crash happened here because the program tried to divide by zero
 (10/0)
- #1 calculate() This function called divide() and passed 0 as the divisor.
- #2 main() The main() function called calculate(10), which eventually caused the
 crash.

CORRECT CODE:

```
#include <stdio.h>

int divide(int a, int b)
{
    return a / b;
}

int calculate(int x)
{
    return divide(x, 2);
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

ERROR7: Debugging Array Out-of-Bounds Errors using GDB

//error7.c (code with out of bound errors)

```
#include <stdio.h>

int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)                //line with error
    {
        scanf("%d", &a[i]);                //Line 8
    }
    return 0;
}
```



```

(kali㉿kali)-[~/Desktop/exp3]
$ vi error7.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error7.c -o error7

(kali㉿kali)-[~/Desktop/exp3]
$ gdb ./error7
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./error7 ...
(gdb) break 8
Breakpoint 1 at 0x114a: file error7.c, line 8.
(gdb) run
Starting program: /home/kali/Desktop/exp3/error7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error7.c:8
8      scanf("%d", &a[i]); //Line 8
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd10
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd20
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd24

```

The error occurred due to wrong loop condition.

CORRECT LOOP CONDITON:

```
for(i = 0; i < 5; i++)
```

```
#include <stdio.h>

int main()
{
    int a[5];

    int i;

    for(i = 0; i < 5; i++)                //error resolved
    {
        scanf("%d", &a[i]);
    }

    return 0;
}
```

ERROR8: Segmentation Faults (SIGSEGV) using GDB

//error8.c (code with segmentation faults)

```
#include <stdio.h>

int main()
{
    int *p = NULL;                        // Pointer points to address 0x0

    *p = 10;                             // Attempting to write to address 0x0 causes Segmentation Fault

    printf("%d\n", *p);

    return 0;
}
```



```

(kali㉿kali)-[~/Desktop/exp3]
$ vi error8.c

(kali㉿kali)-[~/Desktop/exp3]
$ cc -g error8.c -o error8

(kali㉿kali)-[~/Desktop/exp3]
$ gdb ./error8
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./error8...
(gdb) run
Starting program: /home/kali/Desktop/exp3/error8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at error8.c:5
5      *p = 10;           // Attempting to write to address 0x0 causes Segmentation Fault
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
✗ Cannot access memory at address 0x0

```

The fault occurred because the pointer is initialized to NULL. NULL does not refer to valid memory. Dereferencing (*p) attempts to access invalid memory. The operating system detects the illegal access. The program is terminated with a **Segmentation Fault**.

CORRECT CODE:

```

#include <stdio.h>

int main()
{
    int x;

    int *p = &x;           // Assign p to point to valid memory address of x

    *p = 10;               // Valid write

    printf("%d\n", *p);

    return 0;
}

```

5. Systematic Debugging Strategy

Debugging is the process of finding and fixing errors in a program. A systematic approach helps identify bugs quickly and ensures the program works correctly.

- **Observe Expected vs. Actual Output**

First, compare the output you expected with the output the program actually produces. This helps you understand whether the program is working correctly and where the problem might be.

Example:

Expected Output: Square = 25

Actual Output: Square = 10

- **Reproduce the Issue Consistently**

Run the program several times with the same input to make sure the error occurs every time. A bug that can be reproduced consistently is easier to investigate and fix.

Example:

If entering 10 always causes the program to crash, the issue is reproducible.

- **Compile with Flags (-Wall -Wextra -g)**

Compile the program using warning and debugging flags.

-Wall → Shows common warnings.

-Wextra → Shows additional warnings.

-g → Adds debugging information for GDB.

Example:

```
gcc -Wall -Wextra -g program.c -o program
```

These flags help detect mistakes before running the program.

- **Isolate Suspicious Code Sections**

Identify the part of the code that may be causing the problem. Focus on functions or statements related to the incorrect output instead of checking the entire program.

Example:

If the wrong result comes from a `divide()` function, inspect only that function first.

- **Inspect the Program State (printf or GDB Breakpoints)**

Check the values of variables while the program is running. You can use `printf()` statements or GDB breakpoints to understand what is happening inside the program.

Example:

```
printf("Value of x = %d\n", x);
```

or in GDB:

```
(gdb) break main
```

```
(gdb) print x
```

- **Apply a Single Minimal Fix**

Make only one small change at a time. This makes it easier to determine whether the change solved the problem without introducing new bugs.

Example:

Change:

`a + b`

to

`a * b`

instead of making multiple changes at once.

- **Recompile and Re-test**

After fixing the code, compile the program again and run it with the same inputs to check whether the error has been resolved.

Example:

Compile:

```
gcc -Wall -Wextra -g program.c -o program
```

Run:

```
./program
```

- **Verify with Edge Cases and Boundary Inputs**

Finally, test the program using special or extreme input values to ensure it works correctly in all situations.

Examples:

- 0

- Negative numbers (-5)

- Very large numbers (1000000)

- Boundary values (first and last valid array indices)

This helps confirm that the program is reliable and free from hidden bugs.

6. Exercises & Homework Questions

Error Classification

a) Syntax Error

A syntax error occurs when the program does not follow the grammar rules of the C language.

Example:

```
#include <stdio.h>

int main()
{
    int a = 10    // Missing semicolon
    printf("%d", a);
    return 0;
}
```

Reason: Missing ; after int a = 10.

b) Compilation (Type) Error

A compilation or type error occurs when incompatible data types are used or variables are not declared properly.

Example:

```
#include <stdio.h>

int main()
{
    int *p = "Hello";
    return 0;
}
```

```
}
```

Reason: A string is assigned to an integer pointer, causing a type mismatch.

c) Linker Error

A linker error occurs when the compiler cannot find the definition of a function.

Example:

```
#include <stdio.h>
```

```
void display();
```

```
int main()
```

```
{
```

```
    display();
```

```
    return 0;
```

```
}
```

Reason: The function display() is declared but never defined.

d) Runtime Error

A runtime error occurs while the program is executing.

Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10;
```

```
    int b = 0;
```

```
printf("%d", a / b);
```

```
return 0;
```

```
}
```

Reason: Division by zero causes the program to crash.

e) Logical Error

A logical error occurs when the program runs successfully but produces incorrect results.

Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10, b = 5;
```

```
    printf("%d", a + b); // Should be a * b
```

```
    return 0;
```

```
}
```

Reason: The program adds instead of multiplying.

GNU Concept

GDB (GNU Debugger) is one of the tools developed under the GNU Project. It helps programmers find and fix errors in their programs. With GDB, developers can execute a program step by step, inspect variable values, set breakpoints, and analyze program crashes.

Relationship:

GNU provides many software development tools such as GCC (compiler) and GDB (debugger), making GDB an important part of the GNU ecosystem.

Shell Context

Shell

A Shell is a command-line program that allows users to interact with the operating system. It accepts user commands and sends them to the kernel for execution.

Kernel

The Kernel is the core part of the operating system. It manages system resources such as memory, CPU, files, and hardware devices, and acts as a bridge between software and hardware.

Three Common Linux Shells

1. Bash (Bourne Again Shell) – The default shell on most Linux distributions.
2. Zsh (Z Shell) – Offers advanced features such as auto-completion and customization.
3. Fish (Friendly Interactive Shell) – A user-friendly shell with syntax highlighting and command suggestions.

Difference Between Breakpoint, Watchpoint, Catchpoint, and Tracepoint

Features	Breakpoint	Watchpoint	Catchpoint	Tracepoint
Purpose	Stops execution at a specific line of code.	Stops when the value of a variable changes.	Stops when a specific event or exception occurs.	Records information without stopping the program.
Used for	Debugging program flow.	Tracking unexpected variable changes.	Detecting signals, exceptions, or system events.	Monitoring program execution with minimal interruption.
Execution	Program pauses.	Program pauses when the watched variable changes.	Program pauses when the specified event happens.	Program continues running while data is collected.
Example	Pause at line 20.	Stop when count changes.	Stop when a segmentation fault occurs.	Record function calls during execution without pausing.